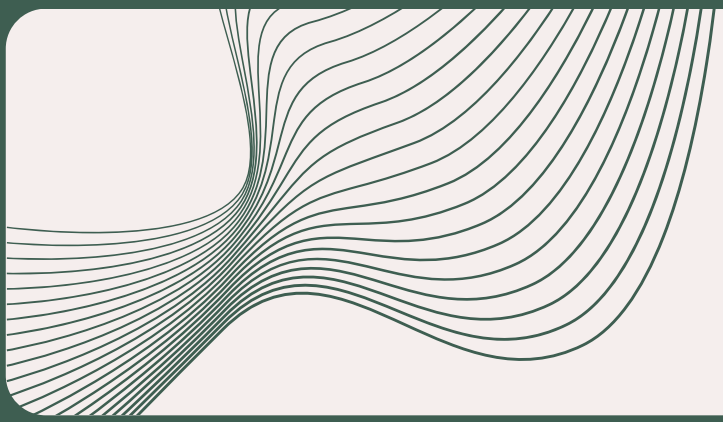
Devixo Solutions

INVENTORY MANAGEMENT SYSTEM

C++ Programming Internship
TASK 03 — PROJECT REPORT
Submitted By: Ayesha Raheela
Internship Domain: C++ Programming
Organization: Devixo Solutions
Project Type: Console-Based Application
Programming Language: C++
Development Environment: Code Utility Online C++ Compiler
Submission Date: 27 October 2026

Project Documentation and Implementation Report



Devixo Solutions
Internship

Table of Contents

1. Introduction — Page 3
2. Project Objective---Page 4
3. Problem Statement and Project Scope — Page 5-6
4. Technologies Used — Page 7
5. Implementation Methodology — Page 8
6. Program Structure and Data Representation — Page 9
7. Code Screenshots ---Page 10-13
8. Implementation and Code Explanation — Page 14-17
9. File Handling and Data Persistence — Page 18-19
10. Input Validation and Error Handling — Page 20
11. Testing and Results — Page 21
12. Challenges Faced and Solutions — Page 22-23
13. Learning Outcomes — Page 24
14. Conclusion — Page 25
15. Project Summary --- Page 26
16. GitHub Repository and Project Details — Page 27
17. Final Statement — Page 28

Introduction

The Inventory Management System is a console-based application developed in C++ as part of Devixo Solutions Internship Task 03.

The purpose of this project is to demonstrate the practical application of C++ programming concepts by developing a system that manages product records, monitors stock levels, calculates inventory value, and processes sales transactions through a menu-driven interface.

The application provides operations for adding new products, viewing inventory records, searching for products by ID, updating product information, deleting records, generating low-stock alerts, and preparing inventory reports. It also includes sales receipt generation and inventory quantity updates after a sale.

The system uses Object-Oriented Programming (OOP) concepts, including classes, inheritance, encapsulation, and polymorphism. It also demonstrates the use of functions, STL vectors, and file input/output operations.

The application stores inventory records in inventory_data.txt and sales receipt information in sales_receipts.txt, supporting file-based data storage.

This project provides practical experience in designing a structured application, managing product information, implementing inventory calculations, processing transactions, and testing software functionality.



Inclusivity



Transparency



Accountability



Project Objective

The primary objective of this project is to develop a functional Inventory Management System using C++.

The specific objectives are:

- To create a menu-driven console application for inventory management.
- To implement operations for adding, viewing, searching, updating, and deleting products.
- To use classes and objects to organize product information and inventory operations.
- To demonstrate inheritance and polymorphism through product-related classes.
- To manage product records using STL vectors.
- To implement file handling for saving and loading inventory data.
- To monitor stock quantities and identify low-stock products.
- To calculate total inventory value and generate inventory reports.
- To generate sales receipts and update inventory quantities after sales.
- To test the application's functionality and verify its behavior.

PROBLEM STATEMENT AND SCOPE

Problem Statement

Businesses need an organized way to maintain product information, monitor available stock, calculate inventory value, and record sales transactions.

Managing these operations manually can become time-consuming and may lead to inaccurate stock records, difficulties locating products, and errors when calculating inventory quantities or values.

The objective of this project is to address these basic requirements by developing a C++ application that allows users to manage product records and process sales through a simple console menu.

The system organizes product information into individual records and provides operations for maintaining inventory, identifying low-stock products, generating inventory reports, and updating quantities after sales.

It also uses file handling to store inventory information and sales receipts, provided the relevant files can be accessed successfully.



Project Scope and Features

The application includes the following features:

1. Add Product

Allows the user to enter product information, including its ID, name, price, and quantity, according to the fields implemented in the program.

2. View Product

Displays the product records maintained by the application, allowing users to review inventory information.

3. Search Product

Searches for a product using its unique ID and displays the matching record when found.

4. Update Product

Allows the user to modify the information associated with an existing product.

5. Delete Product

Removes a selected product record from the inventory when the requested product exists.

6. Low-Stock Alerts

Identifies products whose quantities meet the program's low-stock condition, helping users recognize products that may need restocking.

7. Inventory Reports

Generates a summary of inventory information, including total products, available stock, and total inventory value.

8. Sales Reciept Generation

Processes sales transactions and records the relevant transaction information in sales_receipts.txt, provided the receipt file can be written successfully.

9. Automatic Stock Updates

Deducts the sold quantity from the available inventory after a valid sale.

TECHNOLOGIES USED

The project was developed using the following technologies and C++ programming concepts.

1

Programming Language

C++
C++ was used to implement the application logic, define product-related clesses, manage inventory records, perform calculations, process sales, and handle file operations.

2

Development Environment

CodeUtility Online C++ Compiler
The program was developed and tested in an online C++ environment that provided a source-code editor, execution console, and workspace for project files.

3

Header Files and Libraries

The program uses the following standard C++ headers:

- iostream — for console input and output.
- string — for storing and processing product names and other textual information.
- vector — for maintaining a dynamically sized collection of product objects.
- fstream — for reading and writing inventory and sales files.
- sstream — where used, for processing text records.
- Other standard headers required for calculations, formatting, or input processing.

4

Programming Concepts

The implementation demonstrates:

- Classes and objects.
- Encapsulation.
- Inheritance.
- Polymorphism and virtual functions.
- Functions and modular programming.
- STL vectors.
- Loops and conditional statements.
- Inventory calculations.
- File handling and data persistence.
- Menu-driven programming.

These concepts help organize the program and make its functionality easier to understand and maintain.

IMPLEMENTATION METHODOLOGY

The application was developed in stages to ensure that each major operation could be implemented and tested independently.

Step 1: Define the Product Class
A base class named Product provides the common product interface and shared product information. It also defines virtual functions for product display and value calculation.
Step 2: Implement the StockProduct Class
The StockProduct class inherits from Product and represents a product managed through its stock quantity. It implements the product-specific behavior required by the application.
Step 3: Create the Inventory Manager
The InventoryManager class manages the product collection and provides the operations required for inventory management.
Step 4: Implement Product Management Functions
Separate functions implement adding, viewing, searching, updating, and deleting product records.
Step 5: Implement Stock Monitoring
The program checks product quantities against the low-stock threshold defined in the implementation and identifies products requiring attention.
Step 6: Develop Inventory Reports
The inventory reporting functionality calculates and displays the product count, available stock, and total inventory value.

IMPLEMENTATION METHODOLOGY

Step 7: Implement Sales Processing
The sales functionality validates the requested sale according to the implemented logic, deducts the sold quantity from available stock, and generates a sales receipt.
Step 8: Implement File Handling
The program saves and loads inventory records through inventory_data.txt and writes sales receipt information to sales_receipts.txt.
Step 9: Develop the Menu System
The main execution function displays the available operations and processes the user's selection until the exit option is selected.
Step 10: Test the Application
The program is tested by selecting the available menu operations and checking the resulting output, inventory changes, and file operations.
The staged approach helps organize development and makes individual features easier to examine and test.

PROGRAM STRUCTURE AND DATA REPRESENTATION

The application uses an object-oriented design in which each class has a specific responsibility.

1. Product Class

The Product class represents the common interface for products. It contains shared product information and declares virtual functions such as display() and getValue(). A virtual destructor is also used to support safe cleanup when derived objects are managed through base-class pointers.

2. StockProduct Class

The StockProduct class inherits from Product and provides the stock-related behavior required by the application.
It implements product display and value calculation and supports the program's low-stock monitoring functionality.

3. InventoryManager Class

The InventoryManager class coordinates the inventory operations. Its responsibilities include managing product records, searching for products, updating and deleting records, generating reports, processing sales, and handling file operations.

4. Vector-Based Storage

The application uses vector<Product*> to maintain a dynamically sized collection of product objects through the common Product interface.
This arrangement demonstrates how polymorphism can be used to manage objects through base-class pointers.

PROGRAM STRUCTURE AND DATA REPRESENTATION

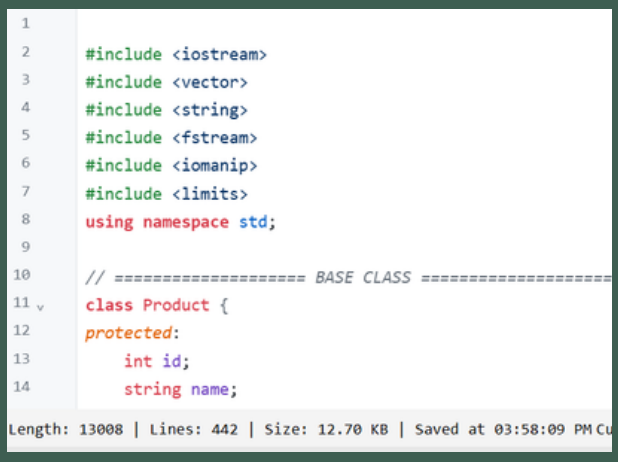
5.Function-Based Organization

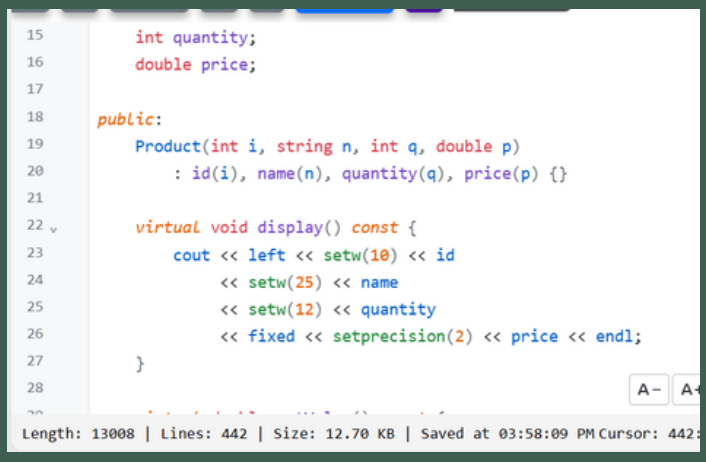
Separate member functions handle individual operations. This structure helps keep product management, stock monitoring, reporting, sales processing, and file handling organized.

6.Main Function

The main() function creates the inventory manager and starts the application by displaying the menu and processing user selections.

Libraries, and Classes





StockProduct and Inventory Manager Classes

Code Screenshots:



```
virtual double getValue() const {
    return quantity * price;
}

virtual ~Product() {}

int getId() const { return id; }
string getName() const { return name; }
int getQuantity() const { return quantity; }
double getPrice() const { return price; }

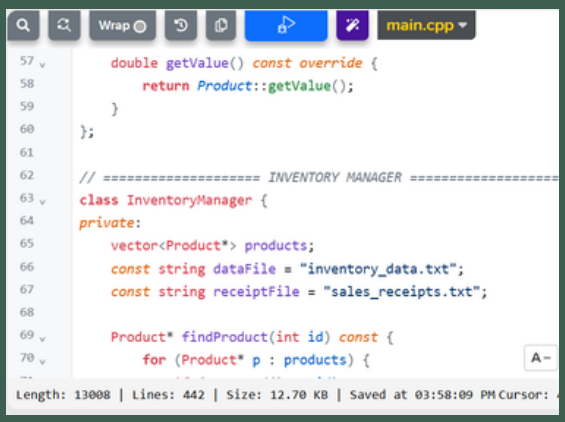
void setName(const string& n) { name = n; }
void setQuantity(int q) { quantity = q; }
void setPrice(double p) { price = p; }
```



```
};

// ==================== DERIVED CLASS ====================
class StockProduct : public Product {
public:
    StockProduct(int i, string n, int q, double p)
        : Product(i, n, q, p) {}

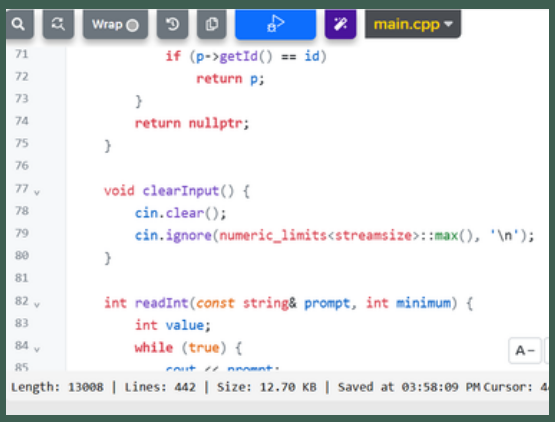
    void display() const override {
        Product::display();
        if (quantity <= 5)
            cout << "  ** LOW STOCK ALERT **" << endl;
    }
```



```
    double getValue() const override {
        return Product::getValue();
    }
};

// ==================== INVENTORY MANAGER ====================
class InventoryManager {
private:
    vector<Product*> products;
    const string dataFile = "inventory_data.txt";
    const string receiptFile = "sales_receipts.txt";

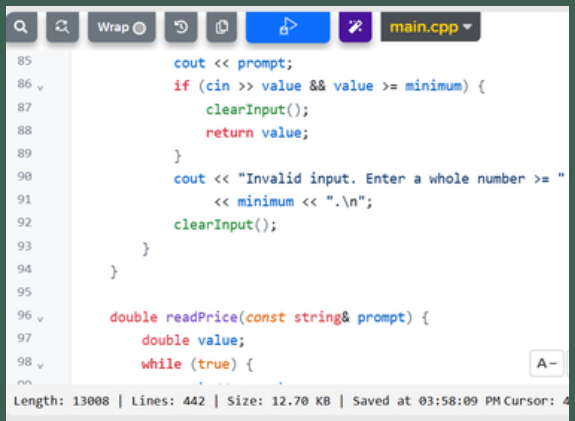
    Product* findProduct(int id) const {
        for (Product* p : products) {
```



```
            if (p->getId() == id)
                return p;
        }
        return nullptr;
    }

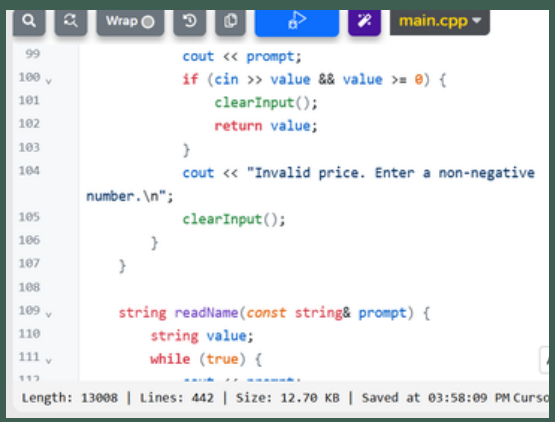
    void clearInput() {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    int readInt(const string& prompt, int minimum) {
        int value;
        while (true) {
```



```
            cout << prompt;
            if (cin >> value && value >= minimum) {
                clearInput();
                return value;
            }
            cout << "Invalid input. Enter a whole number >= "
                 << minimum << ".\n";
            clearInput();
        }
    }

    double readPrice(const string& prompt) {
        double value;
        while (true) {
```



```
            cout << prompt;
            if (cin >> value && value >= 0) {
                clearInput();
                return value;
            }
            cout << "Invalid price. Enter a non-negative
number.\n";
            clearInput();
        }
    }

    string readName(const string& prompt) {
        string value;
        while (true) {
```

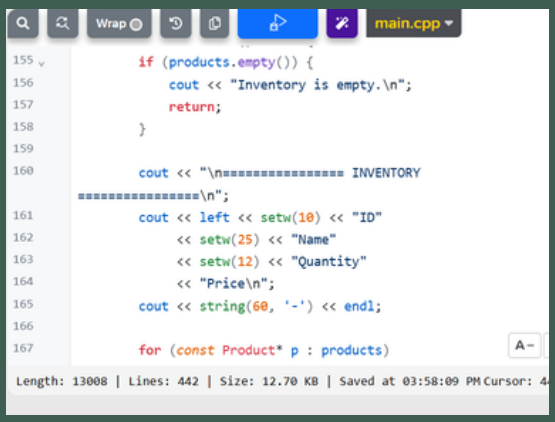
StockProduct and Inventory Manager Classes

Code Screenshots:









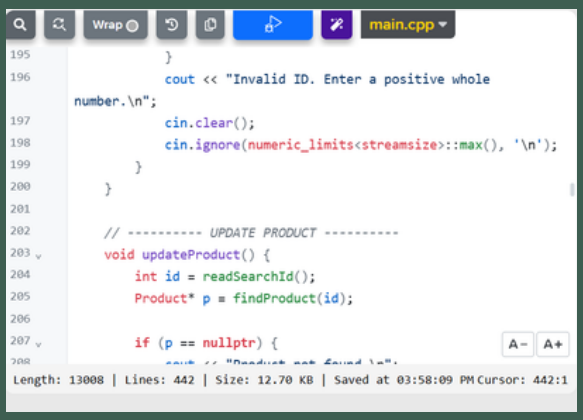


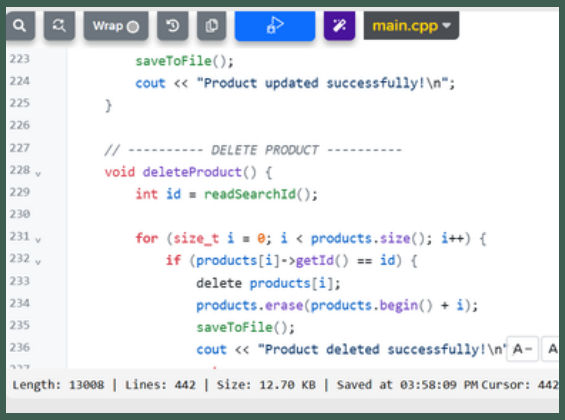


StockProduct and Inventory Manager Classes

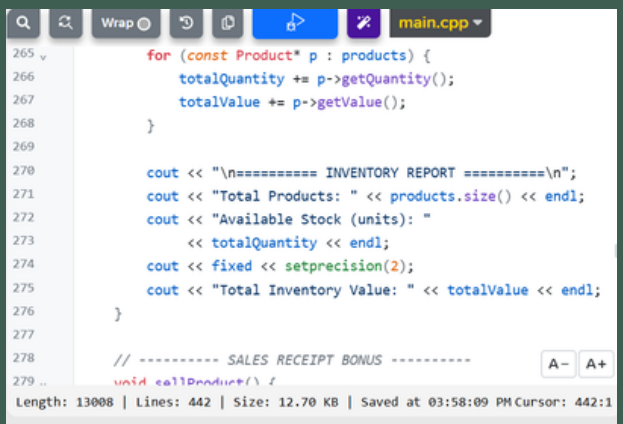
Code Screenshots:

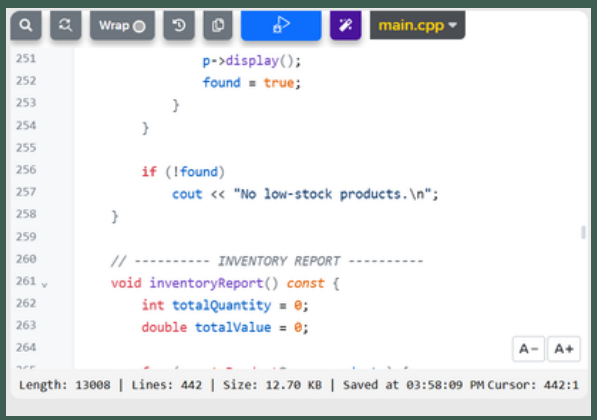








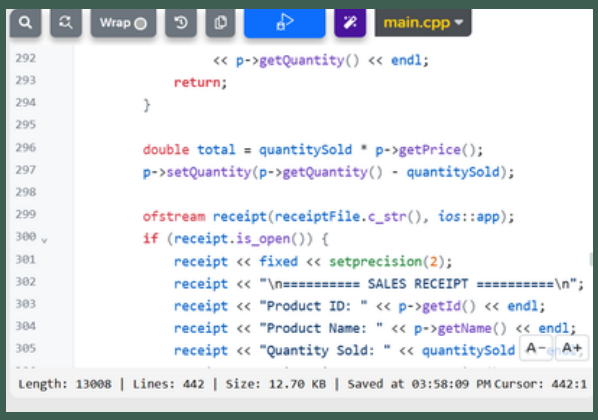


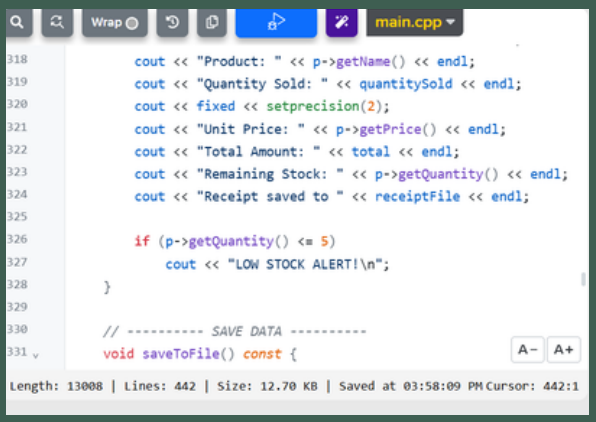


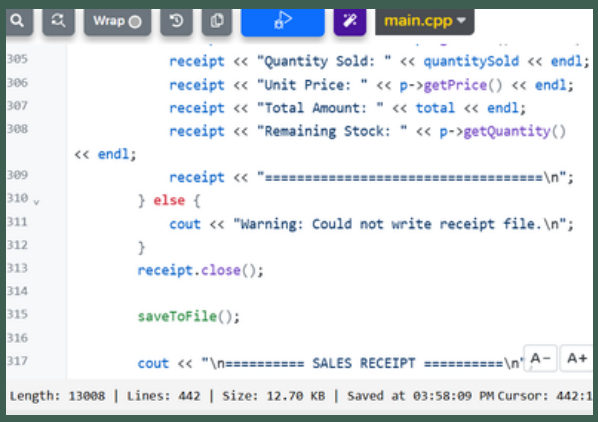
StockProduct and Inventory Manager Classes

Code Screenshots:









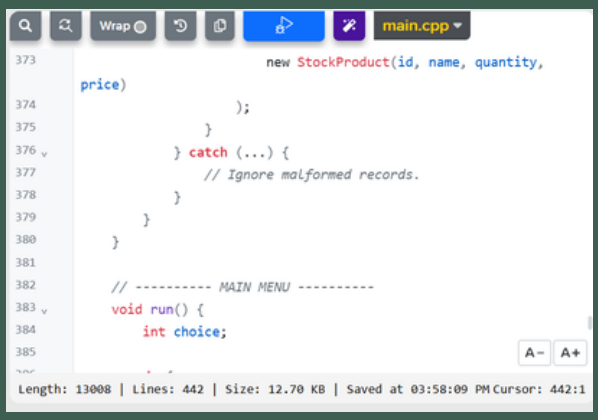


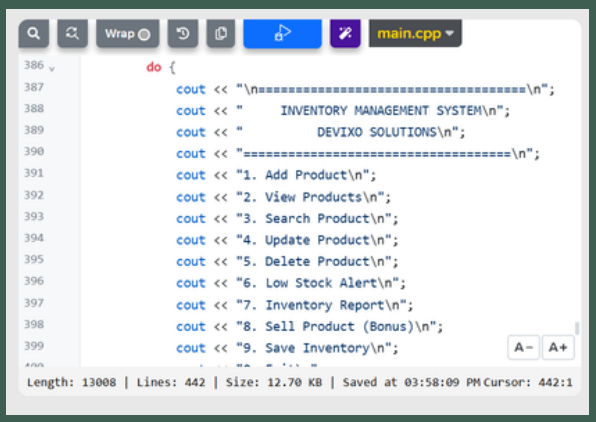


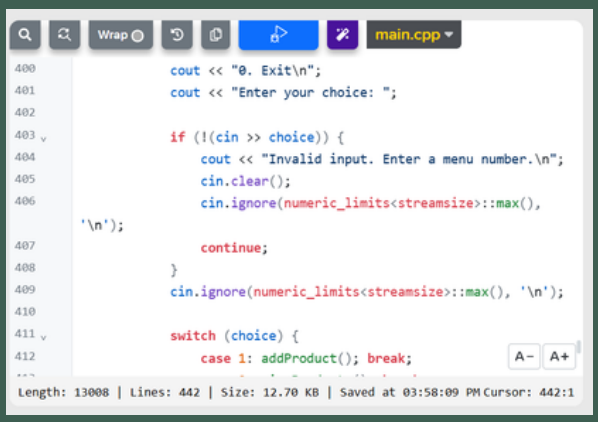
StockProduct and Inventory Manager Classes

Code Screenshots:

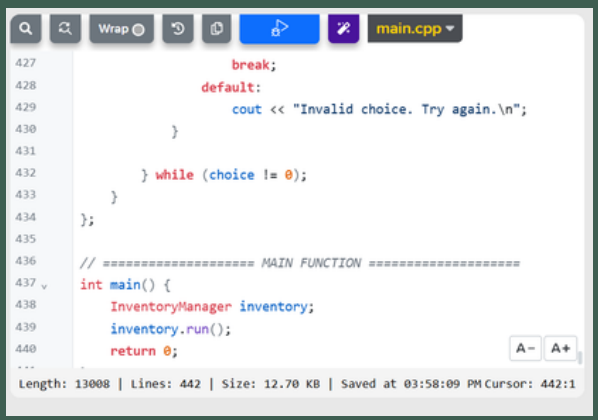












IMPLEMENTATION AND CODE EXPLANATION

The program is organized into separate functions to improve readability, testing, and maintainability.

1. Add Product Function

The product-addition function collects the required information and creates a product record. The program adds the record to the inventory according to the validation and insertion logic implemented in the source code.

```
==================================
      INVENTORY MANAGEMENT SYSTEM
           DEVIXO SOLUTIONS
==================================
1. Add Product
2. View Products
3. Search Product
4. Update Product
5. Delete Product
6. Low Stock Alert
7. Inventory Report
8. Sell Product (Bonus)
9. Save Inventory
0. Exit
Enter your choice: 1
```

```
Enter Product ID: 101
Enter Product Name: Notebook
Enter Quantity: 10
Enter Price: 150
Product added successfully!
```

IMPLEMENTATION AND CODE EXPLANATION

2. View Products Function

The product-viewing function displays the records maintained by the inventory manager. It allows users to review product IDs, names, prices, quantities, and other displayed information.

```
==================================
     INVENTORY MANAGEMENT SYSTEM
          DEVIXO SOLUTIONS
==================================
1. Add Product
2. View Products
3. Search Product
4. Update Product
5. Delete Product
6. Low Stock Alert
7. Inventory Report
8. Sell Product (Bonus)
9. Save Inventory
0. Exit
Enter your choice: 2
```

```
Enter your choice: 2

================ INVENTORY ================
ID        Name                    Quantity    Price
-----------------------------------------------------
101       Notebook                10          150.00
```

3. Search Product Function

The search functionality locates a product using its ID and displays the matching product information when the record exists.

```
==================================
     INVENTORY MANAGEMENT SYSTEM
          DEVIXO SOLUTIONS
==================================
1. Add Product
2. View Products
3. Search Product
4. Update Product
5. Delete Product
6. Low Stock Alert
7. Inventory Report
8. Sell Product (Bonus)
9. Save Inventory
0. Exit
Enter your choice: 3
```

```
Enter your choice: 3
Enter Product ID: 101

Product found:
ID        Name                    Quantity    Price
101       Notebook                10          150.00
```

IMPLEMENTATION AND CODE EXPLANATION

4.Update Product Function

The product-update function locates the selected product and modifies its information according to the user's input.

```
==================================
     INVENTORY MANAGEMENT SYSTEM
          DEVIXO SOLUTIONS
==================================
1. Add Product
2. View Products
3. Search Product
4. Update Product
5. Delete Product
6. Low Stock Alert
7. Inventory Report
8. Sell Product (Bonus)
9. Save Inventory
0. Exit
Enter your choice: 4
```

```
Enter your choice: 4
Enter Product ID: 101
Current details:
101        Notebook                10          150.00
Enter New Product Name: Book
Enter New Quantity: 5
Enter New Price: 180
Product updated successfully!
```

5.Delete Product Function

The product-deletion function removes the selected product from the inventory when the requested record exists.

```
4. Update Product
5. Delete Product
6. Low Stock Alert
7. Inventory Report
8. Sell Product (Bonus)
9. Save Inventory
0. Exit
Enter your choice: 5
Enter Product ID: 103
Product not found.
```

Explanation:
These functions demonstrate how the application maintains and modifies existing inventory information.

IMPLEMENTATION AND CODE EXPLANATION

6.Low-Stock Alert Function

The low-stock functionality identifies products whose quantities meet the configured threshold. In the provided implementation, the threshold is five units or fewer.

```
==================================
     INVENTORY MANAGEMENT SYSTEM
          DEVIXO SOLUTIONS
==================================
1. Add Product
2. View Products
3. Search Product
4. Update Product
5. Delete Product
6. Low Stock Alert
7. Inventory Report
8. Sell Product (Bonus)
9. Save Inventory
0. Exit
Enter your choice: 6
```

```
========== LOW STOCK PRODUCTS ==========
101        Book                        5
  ** LOW STOCK ALERT **
```

7.Inventory Report Function

The inventory report calculates and displays summary information, including the total number of products, available stock, and total inventory value.

```
==================================
     INVENTORY MANAGEMENT SYSTEM
          DEVIXO SOLUTIONS
==================================
1. Add Product
2. View Products
3. Search Product
4. Update Product
5. Delete Product
6. Low Stock Alert
7. Inventory Report
8. Sell Product (Bonus)
9. Save Inventory
0. Exit
Enter your choice: 7
```

```
========== INVENTORY REPORT ==========
Total Products: 1
Available Stock (units): 5
Total Inventory Value: 900.00
```

```
============================
  LIBRARY MANAGEMENT SYSTEM
============================
. Add Book
. View Books
. Search Book
. Update Book Information
. Delete Book
. Issue Book
. Return Book
. Save Data
. Load Data
. Exit
============================
nter your choice: 0
ata saved. Goodbye!
```

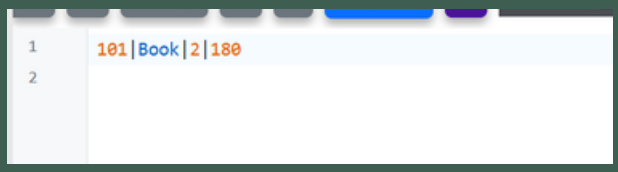
IMPLEMENTATION AND CODE EXPLANATION

8. Sales Processing Function

The sales functionality processes a product sale, deducts the sold quantity from stock, and generates a receipt containing the transaction information.

```
Enter your choice: 8
Enter Product ID: 101
Enter Quantity to Sell: 3

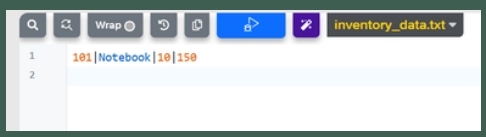
========== SALES RECEIPT ==========
Product: Book
Quantity Sold: 3
Unit Price: 180.00
Total Amount: 540.00
Remaining Stock: 2
Receipt saved to sales_receipts.txt
LOW STOCK ALERT!
```



9. Save Inventory and
10. Exit Menu
11. inventory_data.txt

```
==================================
    INVENTORY MANAGEMENT SYSTEM
         DEVIXO SOLUTIONS
==================================
1. Add Product
2. View Products
3. Search Product
4. Update Product
5. Delete Product
6. Low Stock Alert
7. Inventory Report
8. Sell Product (Bonus)
9. Save Inventory
0. Exit
Enter your choice: 9
Inventory saved successfully.
```

```
==================================
    INVENTORY MANAGEMENT SYSTEM
         DEVIXO SOLUTIONS
==================================
1. Add Product
2. View Products
3. Search Product
4. Update Product
5. Delete Product
6. Low Stock Alert
7. Inventory Report
8. Sell Product (Bonus)
9. Save Inventory
0. Exit
Enter your choice: 0
Inventory saved. Goodbye!
```



FILE HANDLING AND DATA PERSISTENCE

File handling is an important feature of the Library Management System because it allows records to be stored outside the program's temporary memory.
The application uses two text files:

- inventory_data.txt and
- sales_receipts.txt.

1. Inventory Data File

The file inventory_data.txt stores product records for inventory data management.
The program's saving function writes product information to the file, while its loading function reads saved records when the application starts or when loading is requested by the implementation.

2. Inventory Record Format

The inventory data file uses a pipe-separated format. A record may resemble the following, depending on the field order and formatting used in the source code:

101|Notebook|120.00|10
102|Book|180.00|2

The example represents a product ID, product name, unit price, and quantity. The actual order must match the final implementation.

3. Sales Receipt File

The file sales_receipts.txt stores sales receipt information generated by the sales operation.
When a sale is processed, the program prepares the receipt details and attempts to append them to the sales file.
The file provides a text-based record of transactions and demonstrates the use of file output operations.

FILE HANDLING AND DATA PERSISTENCE

4. Saving and Loading Data

The application uses C++ file streams to write and read information.

- ofstream is used for file output.
- ifstream is used for file input.
- File-opening checks help determine whether the relevant operation can proceed.

Successful file persistence depends on the file being written correctly and remaining accessible in the program's working directory.

5. Importance of File Handling

Without file storage, information held only in memory may be lost when the program terminates.

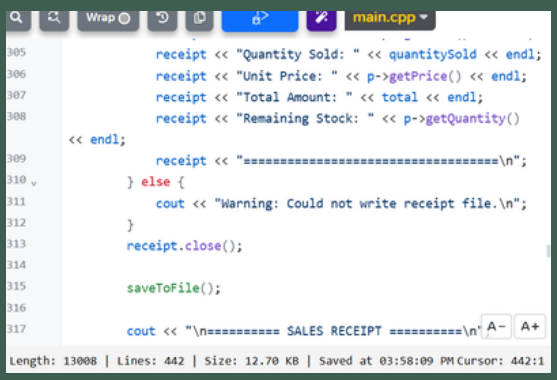
Saving inventory records to a text file helps retain product information between executions when the environment supports persistent storage.

6. Online Compiler Considerations

Online compilers may use temporary or persistent workspaces. Therefore, successful file operations during one run do not guarantee that records will remain available indefinitely.

The source code and project documentation should also be maintained in the GitHub repository.





FILE HANDLING AND DATA PERSISTENCE






```
==================================
     INVENTORY MANAGEMENT SYSTEM
          DEVIXO SOLUTIONS
==================================
1. Add Product
2. View Products
3. Search Product
4. Update Product
5. Delete Product
6. Low Stock Alert
7. Inventory Report
8. Sell Product (Bonus)
9. Save Inventory
0. Exit
Enter your choice: 7
```

```
========== INVENTORY REPORT ==========
Total Products: 1
Available Stock (units): 5
Total Inventory Value: 900.00
```

Explanation:
These screenshots provide evidence of the file operations and the observed behavior of the inventory and sales record files.

INPUT VALIDATION AND ERROR HANDLING

Input validation helps the application respond appropriately when users enter invalid information or request operations that cannot be completed.

1. Product Information Validation

Product information must be processed according to the validation rules implemented in the source code. Invalid product details should not be accepted as valid records.

2. Invalid Numeric Input

The application should handle non-numeric input where numeric values are required, according to its input-validation logic.

3. Menu Validation

The menu-driven interface processes the user's selection and directs execution to the corresponding operation. Invalid choices should be handled with an appropriate message.

4. Product Search and Modification

When a user searches for, updates, or deletes a product that does not exist, the application should report that the requested product could not be found.

5. Sales Quantity Validation

Before processing a sale, the program checks the requested quantity according to its implemented logic. A sale exceeding the available stock should be rejected

6. File-Handling Errors

File operations depend on the availability and accessibility of the inventory and sales files.

The program checks whether the relevant file can be opened. If the receipt file cannot be written, the implementation reports the failure rather than silently treating the receipt as saved.

7. Importance of Error Handling

Clear error messages help users understand why an operation could not be completed. Input validation and error handling also make the program easier to test and more reliable under different input conditions.

```
5. Delete Product
6. Low Stock Alert
7. Inventory Report
8. Sell Product (Bonus)
9. Save Inventory
0. Exit
Enter your choice: 5
Enter Product ID: 103
Product not found.
```

TESTING AND RESULTS

The Inventory Management System was tested through its console interface. The purpose of testing was to examine the behavior of the implemented operations and verify the results of common user actions.

Test case	Test case	Expected Result
TC-01	Start the application	Main Menu displayed
TC-02	Add a product	Successfully added
TC-03	View products	Successfully displayed
TC-04	Search for a product	Matching products are displayed
TC-05	Update a product	The selected product’s information is updated
TC-06	Delete a product	The selected product is removed
TC-07	Generate a low stock report	products meeting the low-stock condition are displayed

TESTING AND RESULTS

The Inventory Management System was tested through its console interface. The purpose of testing was to examine the behavior of the implemented operations and verify the results of common user actions.

Test case	Test case	Expected Result
TC-08	Generate a inventory report	product count, available stock, and inventory value are reported
TC-09	Process a sale	the sold quantity is deducted from available stock
TC-10	Geneartes a sales receipt	Reciept information is written when the file operation succeeds
TC-11	Save inventory data	Product records are written to inventory file
TC-12	Load Inventory data	Saved product records are loaded when available
TC-14	Sell more than available stock	The sale is rejected
TC-15	Enter invalid input	Input is handled according to validation logic

CHALLENGES FACED AND SOLUTIONS

Developing and testing an inventory application involves several practical considerations.

1. Managing Multiple Product Records

Challenge: The application must maintain a collection of product records while supporting additions, updates, searches, and deletions.

Solution: A vector of product pointers was used to manage a dynamically sized collection through the common product interface.

2. Organizing Product Functionality

Challenge: Combining product data, inventory operations, and sales processing in a single large section of code can make the application difficult to understand and maintain.

Solution: The application was organized using the Product, StockProduct, and InventoryManager classes, with separate functions for individual operations.

3. Applying Inheritance and Polymorphism

Challenge: Using common product functionality while supporting product-specific behavior requires a suitable class structure.

Solution: A base class and a derived class were used, with virtual functions supporting polymorphic display and value calculation.

4. Monitoring Stock Levels

Challenge: Users need to identify products whose remaining quantities are low.

Solution: A low-stock report was implemented using the threshold defined in the program.

CHALLENGES FACED AND SOLUTIONS

5. Calculating Inventory Value

Challenge: Preparing an inventory report requires calculating the value of product stock and combining the individual values.

Solution: Product value calculations and an inventory reporting function were used to summarize the value of the inventory.

6. Processing Sales Transactions

Challenge: A sale must update inventory quantities and record transaction details.

Solution: The sales operation was designed to deduct the sold quantity from available stock and attempt to write receipt information to sales_receipts.txt.

7. File Persistence in an Online Environment

Challenge: An online compiler may use temporary storage, affecting whether files remain available across executions.

Solution: The inventory and receipt files were checked during testing. Claims of persistence were limited to the behavior actually verified in the workspace.

Working through these considerations improved my understanding of object-oriented design, inventory calculations, file handling, and systematic testing.

LEARNING OUTCOMES

This project provided practical experience in applying C++ programming concepts to an inventory management application.

1. Understanding Object-Oriented Programming

I gained practical experience in organizing product information and inventory operations using classes and objects.

2. Applying Inheritance

The project helped me understand how a derived class can reuse functionality from a base class while providing additional behavior.

3. Understanding Polymorphism

Using virtual functions demonstrated how product operations can be accessed through a common interface.

4. Working with Vectors

The project provided experience in maintaining a dynamically sized collection of product objects through STL vectors.

5. Implementing File Handling

I gained practical experience in saving and loading inventory information and writing sales receipt records through C++ file streams.

6. Developing Inventory Calculations

The inventory report helped me apply arithmetic operations to calculate product values and summarize stock information.

7. Processing Sales Transactions

The sales functionality provided experience in validating a requested quantity, updating available stock, and recording transaction information.

LEARNING OUTCOMES

8. Handling User Input

The project highlighted the importance of validating input and responding appropriately to unsuccessful operations.

9. Testing and Debugging

Testing individual menu operations helped me examine program behavior, identify potential issues, and verify expected results.

10. Technical Documentation

Preparing the source code, README, screenshots, and project report provided experience in organizing and documenting a software project for review.

Overall, the project helped connect theoretical C++ concepts with practical software development and strengthened my understanding of object-oriented programming.

CONCLUSION

The Inventory Management System was developed as part of Devixo Solutions Internship Task 03 using C++.

The application provides a menu-driven approach to managing product records and includes functionality for adding, viewing, searching, updating, and deleting products. It also supports low-stock monitoring, inventory reporting, sales receipt generation, and stock updates after sales.

The project demonstrates the practical application of classes, objects, encapsulation, inheritance, polymorphism, virtual functions, vectors, functions, and file handling.

Testing the application provided an opportunity to examine its main operations, review the resulting output, and check the behavior of inventory and sales file operations in the development environment.

The project strengthened my understanding of object-oriented design, data management, inventory calculations, transaction processing, testing, and technical documentation.

This task represents a valuable step in my learning journey as a computer science student and provides a foundation for developing more advanced inventory and business management applications in the future.



Project Summary

Items	Details
Organization	Devixo Solutions
Internship Domain	C++ Programming
Task Number	Task 03
Project Title	Inventory management system
Programming Language	C++
Development Environment	CodeUtility online C++ compiler
Main Source File	main.cpp
Inventory Data File	inventory_data.txt
Sales Reciept File	sales_reciept.txt
Documentation	Project report in PDF format

GITHUB REPOSITORY AND PROJECT DETAILS

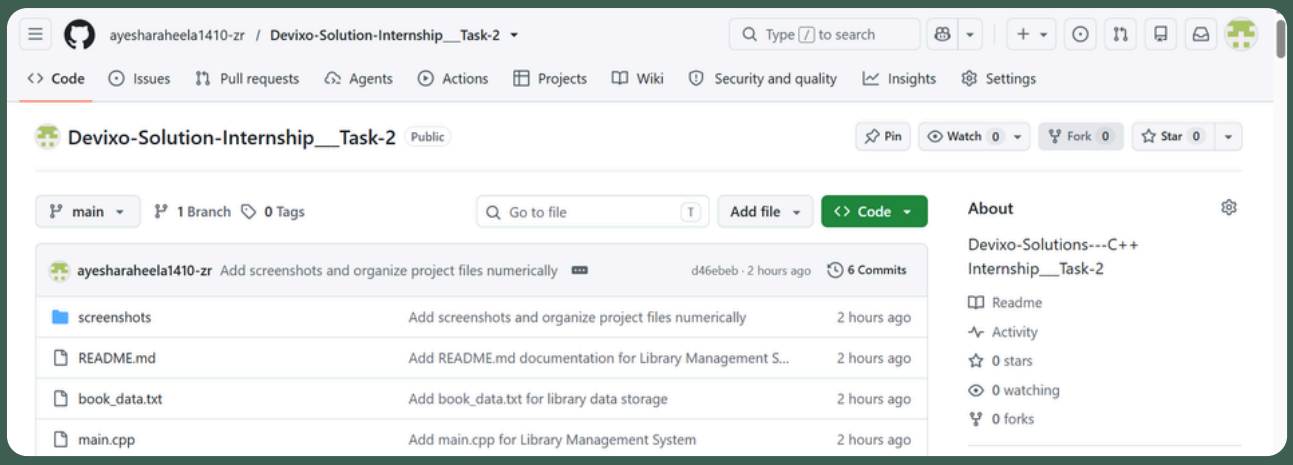
The project source code and documentation are maintained in a GitHub repository.

Repository Name: Devixo-Inventory-Management-System
Repository Link:

https://github.com/ayesharaheela1410-zr/Devixo-Inventory-Management-System

The repository contains the project source code and supporting documentation. Depending on the files uploaded, it may include:

- main.cpp — C++ source code.
- inventory_data.txt — inventory data file.
- sales_receipts.txt — sales receipt file.
- README.md — project overview and usage instructions.
- screenshots/ — screenshots documenting the application.
- Devixo_Inventory_Management_System_Report.pdf — final project report.



Final Statement



I am grateful for the opportunity to apply my programming knowledge to a practical project as part of the Devixo Solutions C++ Internship.

This task helped me strengthen my understanding of object-oriented programming, record management, file handling, testing, and software documentation.

The experience has encouraged me to continue developing my technical skills and work toward building more structured and reliable software applications.

Submitted by: Ayesha Raheela

End of Report

Thank You!